

TEMPERATURÜBERWACHUNG

Technische Dokumentation

Konfiguration, HTTP/HTTPS-Betrieb, Deployment & Sicherheit

Django-Web-Anwendung zur Anzeige von LoRa-Temperatursensoren
Stand: Juli 2026

Inhalt

1 Die Konfiguration (`settings.py`)

- 1.1 Zwei Settings-Module
- 1.2 SECRET_KEY und DEBUG
- 1.3 ALLOWED_HOSTS
- 1.4 CSRF-Schutz und CSRF_TRUSTED_ORIGINS
- 1.5 Sessions und Secure-Cookies
- 1.6 Datenbank und Login-Zwang

2 HTTP- und HTTPS-Verbindung

- 2.1 Architektur: Reverse Proxy mit TLS-Termination
- 2.2 Ablauf einer HTTP-Anfrage
- 2.3 Ablauf einer HTTPS-Anfrage
- 2.4 Das CSRF-403-Problem und seine Lösung
- 2.5 Secure-Cookies: der Kompromiss
- 2.6 Die zwei Betriebsmodi

3 Deployment

- 3.1 Komponenten und geteilte Ressourcen
- 3.2 Die `.env`-Datei
- 3.3 Zwei Modi: HTTP und HTTPS
- 3.4 Deployment Schritt für Schritt

4 Das Cross-Site-Scripting-Problem (XSS)

4.1 Was ist (Stored) XSS?

4.2 Die konkrete Schwachstelle

4.3 Was im schlimmsten Fall passiert

4.4 Die Behebung

1 Die Konfiguration (settings.py)

Dieses Kapitel erklärt die sicherheits- und betriebsrelevanten Einstellungen in `mysite/settings.py`. Der Schwerpunkt liegt auf den Einstellungen, die das Zusammenspiel von HTTP, HTTPS und dem CSRF-Schutz steuern.

1.1 Zwei Settings-Module

Das Projekt hat bewusst zwei getrennte Konfigurationen:

Datei	Zweck	Kernmerkmale
<code>mysite/settings.py</code>	Produktion	<code>DEBUG=False</code> , <code>SECRET_KEY</code> aus Umgebungsvariable, Sicherheits-Einstellungen aktiv
<code>mysite/settings_dev.py</code>	Lokale Entwicklung	<code>DEBUG=True</code> , fest hinterlegter Test-Schlüssel, <code>ALLOWED_HOSTS=['localhost', '127.0.0.1']</code>

Lokal startet man mit `python manage.py runserver --settings=mysite.settings_dev` und braucht dadurch keine Umgebungsvariablen. In Produktion (Docker) wird automatisch `mysite.settings` verwendet.

1.2 SECRET_KEY und DEBUG

```
SECRET_KEY = os.environ.get('SECRET_KEY')
DEBUG = False
```

Der `SECRET_KEY` signiert Sessions, CSRF-Token und weitere sicherheitskritische Werte. Er wird ausschließlich aus der Umgebung gelesen und ist **nicht** im Code hinterlegt – so gelangt er nicht in die Versionsverwaltung. Fehlt er, startet die App bewusst nicht.

Wichtig: In Produktion muss ein langer, zufälliger Wert gesetzt werden. Erzeugen mit:

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

`DEBUG=False` ist Pflicht in Produktion: Bei `True` würde Django bei Fehlern interne Details (Quellcode, Einstellungen) im Browser anzeigen.

1.3 ALLOWED_HOSTS

```
ALLOWED_HOSTS = ["*"] # aktuell: Testbetrieb, akzeptiert jeden Host
```

`ALLOWED_HOSTS` ist eine Whitelist der Hostnamen/IP-Adressen, unter denen die App angesprochen werden darf. Django vergleicht den `Host`-Header jeder Anfrage gegen diese Liste; passt er nicht, folgt `400 Bad Request`. Das schützt gegen sogenannte *Host-Header-Angriffe*.

Für den Testbetrieb steht der Wert auf `["*"]` (jeder Host erlaubt). Für den echten Produktivbetrieb ist im Code bereits eine umgebungsgesteuerte Variante vorbereitet (auskommentiert), sodass die erlaubten Hosts über die Variable `ALLOWED_HOSTS` in der `.env` gesetzt werden können, z. B.

```
ALLOWED_HOSTS=192.168.2.65,raspberrypi.local
```

1.4 CSRF-Schutz und `CSRF_TRUSTED_ORIGINS`

CSRF (Cross-Site Request Forgery) ist ein Angriff, bei dem eine fremde Website den angemeldeten Browser des Opfers dazu bringt, ungewollt eine Aktion (z. B. ein Formular-POST) auf unserer Seite auszuführen. Django schützt jeden POST mit einem Token und einer Herkunftsprüfung.

```
CSRF_TRUSTED_ORIGINS = []
```

Anders als bei `ALLOWED_HOSTS` ist hier **kein Wildcard** `"*"` erlaubt – Django verweigert dann sogar den Start. Eine **leere Liste ist trotzdem korrekt**, denn Django prüft bei jedem POST automatisch, ob der `Origin`-Header des Browsers zum aufgerufenen Host passt (**Same-Origin-Abgleich**). Das funktioniert für jede IP und jeden Hostnamen, ohne dass hier etwas eingetragen werden muss. Ein Eintrag ist nur nötig, wenn Formulare von einer *anderen* Domain/einem anderen Port abgeschickt werden.

Die Verbindung zwischen diesem Same-Origin-Abgleich und HTTP/HTTPS ist der Kern von Kapitel 2 – denn hinter einem Reverse Proxy muss Django erst lernen, ob eine Anfrage verschlüsselt ankam.

1.5 Sessions und Secure-Cookies

```
SESSION_EXPIRE_AT_BROWSER_CLOSE = False
SESSION_COOKIE_AGE = 60 * 60 * 24 * 365 # 1 Jahr

_secure_cookies = os.environ.get('SECURE_COOKIES', 'false').strip().lower() == 'true'
SESSION_COOKIE_SECURE = _secure_cookies
CSRF_COOKIE_SECURE = _secure_cookies
```

Die ersten beiden Zeilen sorgen dafür, dass ein Login ein Jahr lang gültig bleibt und nicht beim Schließen des Browsers verfällt.

Die **Secure-Cookies** steuern, ob Session- und CSRF-Cookie nur über HTTPS gesendet werden. Sie werden über die Umgebungsvariable `SECURE_COOKIES` geschaltet (Standard: aus). Warum das ein Schalter und keine feste Einstellung ist, erklärt Abschnitt 2.5.

1.6 Datenbank und Login-Zwang

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
        'OPTIONS': {'timeout': 20},    # bei Sperre bis zu 20 s warten  
    }  
}
```

Da sich drei Prozesse (zwei Web-Worker und der LoRa-Empfänger) eine einzige SQLite-Datei teilen, sorgt `timeout: 20` dafür, dass ein wartender Schreibzugriff bis zu 20 Sekunden auf eine Sperre wartet, statt sofort mit `database is locked` abzubrechen.

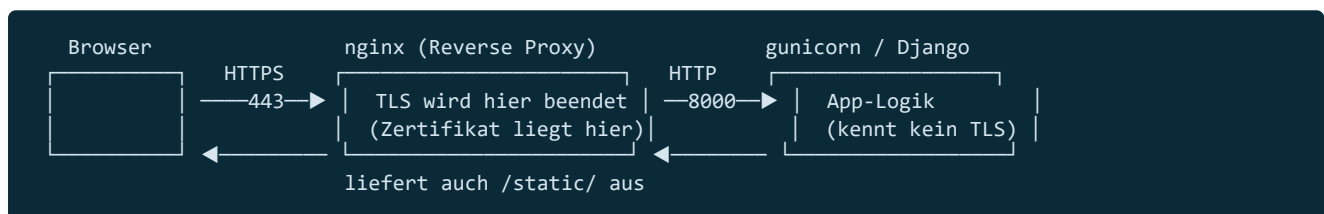
Der Login-Zwang läuft über eine eigene Middleware (`plot/middleware.py`): Jeder Pfad außer `/login/` und `/static/` erfordert eine Anmeldung.

2 HTTP- und HTTPS-Verbindung

Dies ist der wichtigste Teil für den Betrieb. Er erklärt, wie eine Anfrage vom Browser bis zu Django läuft, warum HTTPS ein paar zusätzliche Einstellungen braucht und wie beide Betriebsarten sauber umgeschaltet werden.

2.1 Architektur: Reverse Proxy mit TLS-Termination

Die App läuft nie „nackt“ im Internet. Davor sitzt **nginx** als Reverse Proxy. nginx nimmt die Verbindung des Browsers an und reicht sie intern per einfachem HTTP an den Django-Server (gunicorn) weiter. Bei HTTPS wird die Verschlüsselung **bei nginx beendet** („TLS-Termination“):



Wichtige Folge: **Django selbst sieht immer nur HTTP**, auch wenn der Browser HTTPS benutzt hat. Django weiß also von sich aus nicht, ob die ursprüngliche Verbindung verschlüsselt war. Genau daraus entsteht das Problem in Abschnitt 2.4.

2.2 Ablauf einer HTTP-Anfrage

1. Browser ruft `http://<host>/` auf (Port 80).
2. nginx nimmt an und leitet an `web:8000` weiter; dabei setzt es den Header `X-Forwarded-Proto: http`.
3. Django sieht eine HTTP-Anfrage. Der `Origin`-Header des Browsers lautet `http://<host>` und passt zum aufgerufenen Host → CSRF-Prüfung besteht, Login funktioniert.

2.3 Ablauf einer HTTPS-Anfrage

Damit HTTPS funktioniert, müssen zwei Bausteine zusammenspielen:

a) nginx meldet das ursprüngliche Schema (in `nginx.https.conf`):

```
proxy_set_header X-Forwarded-Proto $scheme; # hier "https"
```

b) Django vertraut diesem Header (in `settings.py`):

```
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
```

Erst dadurch erkennt Django: „Diese Anfrage kam ursprünglich über HTTPS an“ – obwohl sie intern per HTTP eintraf. `request.is_secure()` liefert dann `True`, und der Same-Origin-Abgleich rechnet mit `https://<host>`.

Warum ist das sicher? nginx *überschreibt* den Header bei jeder Anfrage selbst, ein Client kann ihn nicht fälschen. Zusätzlich ist der Django-Container nur über nginx erreichbar (kein direkter Zugriff von außen).

2.4 Das CSRF-403-Problem und seine Lösung

Fehlt Baustein (b), entsteht ein tückischer Fehler: Der Browser sendet bei HTTPS `Origin: https://<host>`, Django hält die Anfrage aber für `http`. Die beiden passen nicht zusammen → Django lehnt **jeden POST, inklusive Login, mit 403 Forbidden ab**. Die App scheint zu laufen, aber niemand kann sich anmelden.

Dieses Verhalten wurde direkt nachgestellt:

Konfiguration	HTTPS-Login-POST
OHNE <code>SECURE_PROXY_SSL_HEADER</code>	403 – Login unmöglich
MIT <code>SECURE_PROXY_SSL_HEADER</code> + <code>X-Forwarded-Proto</code>	200 – Login funktioniert
MIT Header, aber Angriff von fremder Origin	403 – Angriff korrekt blockiert

Beide Bausteine sind im Projekt gesetzt, HTTPS funktioniert also korrekt, sobald die Zertifikate vorliegen.

2.5 Secure-Cookies: der Kompromiss

Ein Cookie mit dem `Secure`-Flag wird vom Browser **nur über HTTPS** gesendet. Das schützt das Session-Cookie davor, über eine unverschlüsselte HTTP-Verbindung mitgelesen und zur Übernahme der Session missbraucht zu werden.

Der Haken: `SESSION_COOKIE_SECURE = True` bedeutet, dass ein Login über `http://` nicht mehr funktioniert – der Browser gibt das Cookie dann nur noch über HTTPS heraus. Deshalb ist es kein fester Wert, sondern der Schalter `SECURE_COOKIES`:

- `SECURE_COOKIES=false` → HTTP-Betrieb möglich (Cookie aber nicht Secure-markiert).
- `SECURE_COOKIES=true` → nur HTTPS; Cookies sind geschützt.

2.6 Die zwei Betriebsmodi

Das Projekt ist bewusst so ausgelegt, dass es **entweder** reines HTTP **oder** reines HTTPS fährt – nie beides gleichzeitig. Statt einer `if`-Bedingung (die nginx für ganze `server`-Blöcke nicht sauber unterstützt) gibt es zwei fertige Konfigurationen, zwischen denen eine Umgebungsvariable umschaltet:

Datei	Verhalten
<code>nginx.http.conf</code>	Nur Port 80, reines HTTP.
<code>nginx.https.conf</code>	Port 80 leitet per <code>301</code> dauerhaft auf HTTPS um; Port 443 bedient die App mit TLS.

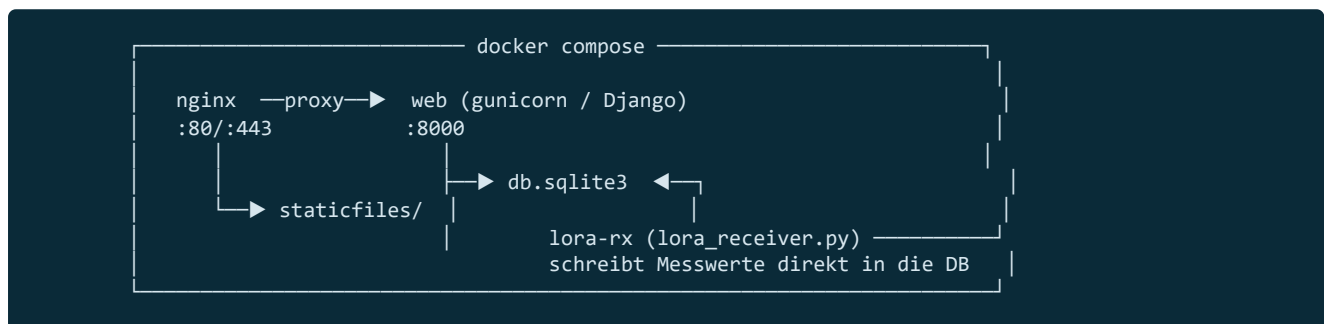
Die Auswahl trifft `NGINX_CONF` in der `.env`; `docker-compose.yml` mountet die passende Datei. Details im nächsten Kapitel.

3 Deployment

Wie die Anwendung mit Docker in Betrieb geht – Komponenten, Konfiguration über die `.env` und die konkreten Schritte für beide Betriebsmodi.

3.1 Komponenten und geteilte Ressourcen

Das Deployment besteht aus drei Docker-Diensten, die sich eine SQLite-Datei und die statischen Dateien teilen:



Dienst	Aufgabe
web	Django über gunicorn (die eigentliche Web-Anwendung).
lora-rx	Eigenständiger LoRa-Empfänger, schreibt Sensordaten per rohem SQLite in dieselbe DB.
nginx	Reverse Proxy, TLS-Termination und Ausliefern der statischen Dateien.

3.2 Die .env-Datei

Die `.env` liegt im Projektwurzel, ist per `.gitignore` ausgeschlossen und wird von `docker compose` gelesen. **Ohne sie bricht bereits `docker compose up` ab** („env file not found“).

Variable	Bedeutung	Aktiv?
SECRET_KEY	Signierschlüssel, Pflicht	ja
SECURE_COOKIES	<code>true</code> = Cookies nur über HTTPS	ja
NGINX_CONF	wählt <code>nginx.http.conf</code> oder <code>nginx.https.conf</code>	ja
ALLOWED_HOSTS	erlaubte Hosts (Komma-Liste)	nur wenn die env-Zeile im Code aktiviert ist

3.3 Zwei Modi: HTTP und HTTPS

Modus HTTP

```
SECRET_KEY=dein-langer-zufallswert
NGINX_CONF=nginx.http.conf      # oder weglassen - HTTP ist Default
SECURE_COOKIES=false
```

Kein Zertifikat nötig. Login läuft über `http://<host>/`.

Modus HTTPS

```
SECRET_KEY=dein-langer-zufallswert
NGINX_CONF=nginx.https.conf
SECURE_COOKIES=true
```

Zusätzlich müssen `certs/server.crt` und `certs/server.key` vorliegen. Der Aufruf über `http://` wird automatisch auf `https://` umgeleitet.

Umschalten zwischen den Modi = eine Zeile in der `.env` ändern und `docker compose up -d` ausführen. Kein Code-Umbau, kein Neu-Bauen des Images.

3.4 Deployment Schritt für Schritt

```
# 1. .env anlegen (siehe oben) und certs/ bereitstellen
mkdir certs          # im HTTPS-Modus die Zertifikate hier ablegen

# 2. Container bauen und starten
docker compose up -d --build

# 3. Datenbank-Migrationen anwenden
docker compose exec web python manage.py migrate

# 4. Statische Dateien einsammeln (CSS/JS für nginx)
docker compose exec web python manage.py collectstatic --noinput

# 5. Login-Benutzer anlegen
docker compose exec web python manage.py createsuperuser
```

Nach jeder Änderung an CSS/JS muss `collectstatic` erneut laufen, sonst liefert nginx den alten Stand aus (z. B. ohne den XSS-Fix aus Kapitel 4).

4 Das Cross-Site-Scripting-Problem (XSS)

Dieses Kapitel erklärt die XSS-Schwachstelle, die im Projekt gefunden und behoben wurde – was sie bedeutet, wie sie funktionierte, was im schlimmsten Fall passiert wäre und wie sie geschlossen wurde.

4.1 Was ist (Stored) XSS?

Cross-Site Scripting (XSS) bezeichnet das Einschleusen von fremdem JavaScript-Code in eine Webseite, der dann im Browser *anderer* Nutzer ausgeführt wird – mit allen Rechten, die diese Nutzer auf der Seite haben.

„**Stored**“ (**persistent**) ist die gefährlichste Variante: Der Schadcode wird serverseitig *gespeichert* und automatisch bei jedem ausgeliefert, der die Seite öffnet – ohne dass das Opfer irgendwo klicken muss.

4.2 Die konkrete Schwachstelle

In der Sensorverwaltung kann jeder angemeldete Nutzer einem Sensor einen freien **Namen** geben. Dieser Name wurde auf dem Dashboard beim Aufbau einer Sensorkarte so eingesetzt:

```
// vorher – UNSICHER
head.innerHTML = '<div class="sensor-id">' + nodeId(sensor.node) + '</div>' +
                '<div class="sensor-name">' + cardName(sensor) + '</div>';
```

Über `innerHTML` wird der eingesetzte Text vom Browser als **HTML interpretiert**. Der Weg des Schadcodes:

```

Sensorverwaltung   Datenbank   JSON-API   Dashboard (JS)
Textfeld "Name"   SensorNodes.name  /temperature-data/  innerHTML = ...Name...
(frei wählbar)    (ungeprüft)      (ungeprüft)         ▲ hier wird HTML ausgeführt
```

Trägt jemand als Namen z. B. Folgendes ein ...

```
<img src=x onerror="/* beliebiger JavaScript-Code */">
```

... dann ist das für `innerHTML` kein Text, sondern ein Bild-Element. Das Laden schlägt fehl (`src=x`), woraufhin der `onerror`-Code ausgeführt wird – bei **jedem**, der das Dashboard öffnet. Da das Live-Dashboard alle 10 Sekunden neu lädt, geschieht das immer wieder.

4.3 Was im schlimmsten Fall passiert

Der eingeschleuste Code läuft im Browser des Opfers **im Kontext von dessen angemeldeter Session** und kann alles tun, was das Opfer darf:

- **Konto-/Admin-Übernahme:** Öffnet ein Django-Superuser das Dashboard, kann der Code dessen Session nutzen, um im Admin-Bereich einen neuen Superuser anzulegen, Passwörter zu ändern oder die gesamte Datenbank auszulesen.

- **Datenabfluss:** Alle Messdaten auslesen und an einen fremden Server senden.
- **Selbstverbreitung:** Weil der Code gespeichert ist, trifft eine einzige böswillige Umbenennung jeden Betrachter erneut, bis der Name bereinigt wird.
- **Phishing:** Eine gefälschte Login-Maske über die Seite legen und Zugangsdaten abgreifen.

Einordnung für dieses Projekt: Die App steht hinter einem Login, ein anonymer Angreifer aus dem Internet scheidet also aus. Das reale Risiko hängt an der Nutzerzahl: Bei einem einzigen, vertrauenswürdigen Nutzer ist es faktisch „Selbst-XSS“. Sobald es mehrere Nutzer gibt oder ein Konto kompromittiert wird, führt ein gepflanzter Name jedoch Code im Browser des Admins aus – mit Ausweitung bis zur vollständigen Übernahme.

4.4 Die Behebung

Die Lösung ist, den Namen nicht als HTML, sondern als reinen Text einzusetzen. Statt String-Bau in `innerHTML` werden die Elemente einzeln erzeugt und über `textContent` befüllt:

```
// nachher – SICHER
var nameEl = document.createElement('div');
nameEl.className = 'sensor-name';
nameEl.textContent = cardName(sensor); // reiner Text, wird nie als HTML interpretiert
```

`textContent` setzt den Wert wörtlich als Text. Ein Name wie `<img src=x onerror=...>` erscheint dadurch als sichtbarer Text auf der Karte, statt ausgeführt zu werden. Layout und Verhalten bleiben identisch.

Grundregel: Von Nutzern stammende Werte niemals per `innerHTML` in die Seite schreiben. In Django-Templates übernimmt das automatische Escaping (`{{ wert }}`) diesen Schutz; im JavaScript ist `textContent` (statt `innerHTML`) das Gegenstück.